Stock Trading Simulator

Milestone 2 Report

Course: Object Oriented Programming (1pm - 2:15pm)

University: IBA Karachi

Prepared By:

Syed Muhammad (32550)
Muhammad Moosa (33601)

Course Instructor: Samreen Kazi

Contents

1 Introduction **2**

2 Project Objective **2**

3 Scope of Milestone 2 Prototype **2**

4 Object Oriented Programming Concepts Used **3**
- 4.1 Encapsulation . . . 3
- 4.2 Inheritance . . . 3
- 4.3 Polymorphism . . . 3
- 4.4 Abstraction . . . 3
- 4.5 Exception Handling . . . 3

5 System Design **3**
- 5.1 Main Classes . . . 3
- 5.2 Data Structures Used . . . 4

6 Risk Checks and Validation **4**
- 6.1 Buy Order Validation . . . 4
- 6.2 Sell Order Validation . . . 4
- 6.3 Exception Handling . . . 4

7 Trade Record and Summary **4**

8 Portfolio Statement **5**

9 UML Class Diagram **5**

10 Test Cases **6**

11 Sample Output / Screenshots **6**

12 GitHub Repository Organization **8**

13 Conclusion **9**

14 Appendix **9**
- 14.1 Project Title . . . 9
- 14.2 Group Members . . . 9
- 14.3 Submitted Files . . . 9

1. Introduction

The **Stock Trading Simulator** is a C++ based Object Oriented Programming project developed to simulate the core working of a stock trading system. The prototype allows a user to place buy and sell orders, add market liquidity, automatically match orders, record executed trades, track portfolio status, and undo the most recent trade.

This submission is an **initial prototype** for Milestone 2. It is not the final complete implementation of the project, but it demonstrates the main architecture, class design, and core interactions required by the system.

2. Project Objective

The objective of this project is to design a stock trading simulator that applies Object Oriented Programming concepts in a practical way. The prototype aims to:

- model buy and sell orders using classes,
- demonstrate inheritance and runtime polymorphism,
- maintain a portfolio containing cash and shares,
- automatically match orders using priority-based logic,
- maintain trade history,
- calculate session summary information,
- support undo functionality,
- perform risk checks before accepting invalid orders.

3. Scope of Milestone 2 Prototype

The current system is a prototype only. It demonstrates the basic class structure and major interactions, but it is not intended to be the final full-scale system at this stage.

The following features are included in the current prototype:

- User buy order placement
- User sell order placement
- Market buy liquidity addition
- Market sell liquidity addition
- Order matching based on price priority
- Executed trade recording
- Trading session summary
- Portfolio statement display

- Undo last trade
- Exception-based risk checks
- Menu-driven interface

4. Object Oriented Programming Concepts Used

4.1 Encapsulation

Each class stores its data members privately and provides public member functions to access or modify them in a controlled way.

4.2 Inheritance

The project uses a base abstract class `Order` and two derived classes `BuyOrder` and `SellOrder`.

4.3 Polymorphism

The `Order` class contains a pure virtual function `getType()`, which is overridden in derived classes. This demonstrates runtime polymorphism.

4.4 Abstraction

The `Order` class is abstract and defines the common structure for all order types without being instantiated directly.

4.5 Exception Handling

Custom exceptions are used to reject invalid buy and sell orders and to prevent undo operations when no trade exists.

5. System Design

5.1 Main Classes

1. **Order**
   Abstract base class storing common order information such as order ID, price, quantity, timestamp, and whether the order belongs to the user.
2. **BuyOrder**
   Derived from `Order`. Represents a buy order in the system.
3. **SellOrder**
   Derived from `Order`. Represents a sell order in the system.
4. **Portfolio**
   Stores and manages the user's financial information including available cash, reserved cash, owned shares, reserved shares, equity, and profit/loss.

5. **TradeRecord**
   Stores the details of an executed trade such as timestamp, buy order ID, sell order ID, trade price, and quantity.

6. **UndoRecord**
   Stores the information needed to reverse the most recent executed trade.

7. **TradingSystem**
   The central class responsible for handling order books, matching logic, trade execution, trade history, session summary, and interaction with the portfolio.

8. **Custom Exceptions**
   Includes `TradingException`, `InsufficientFundsException`, `InsufficientSharesException`, and `UndoException`.

5.2 Data Structures Used

- **Priority Queue** for buy and sell order books
- **Vector** for storing trade history
- **Stack** for undo operations

6. Risk Checks and Validation

To maintain system integrity, the prototype performs validation before accepting orders.

6.1 Buy Order Validation

A buy order is accepted only if the user has enough cash balance available to reserve the required amount.

6.2 Sell Order Validation

A sell order is accepted only if the user owns enough shares to sell the requested quantity.

6.3 Exception Handling

If an order is invalid, the system throws a custom exception instead of allowing inconsistent data.

7. Trade Record and Summary

The prototype maintains a complete record of all executed trades. Each trade record stores:

- timestamp,
- buy order ID,
- sell order ID,

- traded price,
- quantity.

The prototype also generates a trading session summary including:

- total trading volume,
- average traded price,
- total number of executed trades.

8. Portfolio Statement

The portfolio statement generated by the system displays:

- current cash balance,
- reserved cash,
- shares owned,
- reserved shares,
- last trade price,
- total equity,
- overall profit or loss.

9. UML Class Diagram

The UML class diagram for the current prototype is shown below.

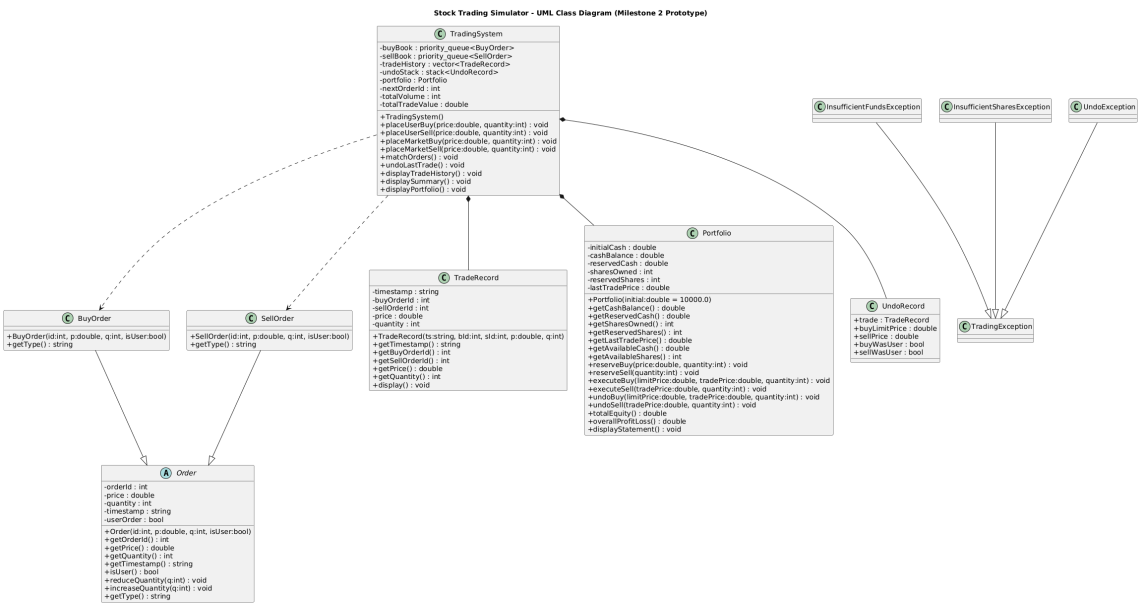

Figure 1: UML Class Diagram of Stock Trading Simulator Prototype

10. Test Cases

The following test cases cover the major functionalities currently implemented in the prototype.

Test Case ID	**Feature**	**Input**	**Expected Output**
TC-01	Place valid user buy order	Price = 100, Quantity = 20, enough cash available	Order is accepted and added to buy order book
TC-02	Place valid user sell order	Price = 110, Quantity = 10, enough shares available	Order is accepted and added to sell order book
TC-03	Reject invalid buy order	Very high price and quantity with insufficient cash balance	`InsufficientFundsException` is thrown
TC-04	Reject invalid sell order	Sell quantity greater than available shares	`InsufficientSharesException` is thrown
TC-05	Match compatible orders	User buy order and market sell order with matching prices	Trade is executed, trade history is updated, and portfolio is updated
TC-06	Display trade history	View trade history after one or more successful matches	System shows executed trade records correctly
TC-07	Display session summary	Execute multiple trades and select summary option	Total volume and average traded price are shown correctly
TC-08	Undo last trade	Execute one trade and then choose undo option	Last trade is reversed and portfolio is restored

11. Sample Output / Screenshots

Screenshots of the running prototype can be inserted below.

```
Main Menu
========= Stock Trading Simulator =========
1. Place User Buy Order
2. Place User Sell Order
3. Add Market Buy Liquidity
4. Add Market Sell Liquidity
5. Match Orders
6. View Executed Trades
7. View Session Summary
8. View Portfolio Statement
9. Undo Last Trade
0. Exit
Enter choice:
```

```
========= Stock Trading Simulator =========
1. Place User Buy Order
2. Place User Sell Order
3. Add Market Buy Liquidity
4. Add Market Sell Liquidity
5. Match Orders
6. View Executed Trades
7. View Session Summary
8. View Portfolio Statement
9. Undo Last Trade
0. Exit
Enter choice: 1
Enter buy price: 10000
Enter quantity: 15
Order rejected: Insufficient cash balance.
```

Figure 2: Menu-Driven Interface

```
========== Trading Session Summary ==========
Total Trading Volume : 0
Average Traded Price : 0
Executed Trades      : 0
=============================================

========= Stock Trading Simulator =========
1. Place User Buy Order
2. Place User Sell Order
3. Add Market Buy Liquidity
4. Add Market Sell Liquidity
5. Match Orders
6. View Executed Trades
7. View Session Summary
8. View Portfolio Statement
9. Undo Last Trade
0. Exit
Enter choice: 8

========== Portfolio Statement ==========
Cash Balance     : 9530
Reserved Cash    : 470
Shares Owned     : 100
Reserved Shares  : 10
Last Trade Price : 100
Total Equity     : 20000
Overall P/L      : 10000
=========================================
```

```
========= Stock Trading Simulator =========
1. Place User Buy Order
2. Place User Sell Order
3. Add Market Buy Liquidity
4. Add Market Sell Liquidity
5. Match Orders
6. View Executed Trades
7. View Session Summary
8. View Portfolio Statement
9. Undo Last Trade
0. Exit
Enter choice: 1
Enter buy price: 47
Enter quantity: 10
User Buy Order placed successfully. Order ID: 1

========= Stock Trading Simulator =========
1. Place User Buy Order
2. Place User Sell Order
3. Add Market Buy Liquidity
4. Add Market Sell Liquidity
5. Match Orders
6. View Executed Trades
7. View Session Summary
8. View Portfolio Statement
9. Undo Last Trade
0. Exit
Enter choice: 2
Enter sell price: 55
Enter quantity: 10
User Sell Order placed successfully. Order ID: 2
```

Figure 3: Trade Execution and Portfolio Output

12. GitHub Repository Organization

The project files are organized neatly on GitHub for bonus evaluation. A recommended structure is shown below.

```
Stock-Trading-Simulator/

        README.md
        simulator.cpp

        docs/
              Milestone2_Report.pdf
              uml-diagram.png
              test-cases.pdf
```

GitHub Repository Link: https://github.com/smuh42/Stock-Trading-Simulator

13. Conclusion

This milestone establishes the initial design and working prototype of the Stock Trading Simulator. The system demonstrates the main class structure, use of Object Oriented Programming concepts, order matching, portfolio management, trade history, and exception-based validation.

Although the current implementation is still a prototype and not the final complete system, it provides a strong foundation for future development and satisfies the main requirements of Milestone 2.

14. Appendix

14.1 Project Title

Stock Trading Simulator

14.2 Group Members

- Syed Muhammad (32550)
- Muhammad Moosa (33601)

14.3 Submitted Files

- C++ Source Code
- UML Class Diagram
- Test Cases in PDF
- Milestone 2 Report